

Project Overview (Plain Language)

Project overview, in plain language

What this project is, how it works, and — for every tool involved — why that tool and not something else. Written to be understandable without a technical background; the other docs in this folder go deeper on any one piece.

What this project actually does

Imagine a workers’ comp insurance company gets thousands of medical bills from doctors and clinics every month. Some of those bills have honest mistakes, and a few are deliberately padded. Someone has to check each one — but a human reviewing every single bill line by line doesn’t scale.

This project is a demo of an assistant that does a first pass on that job automatically. You give it a bill, and within a second it tells you: does this bill look OK, or does it show signs of one of four common billing problems? And if something looks wrong, it can also explain *why*, in plain English, citing the actual rule that was broken.

Important: every bill in this project is 100% made up. No real patients, doctors, or insurance claims are used anywhere — see the “Is any of this real?” section below.

The four things it checks for

Problem	In plain terms
Unbundling	Billing two procedures separately when one of them is actually already included in the other — like being charged separately for “a car” and “the engine that comes with the car.”
Fragmented billing	Billing the same procedure code way more times than is medically realistic in one day.
Causality mismatch	The treatment billed doesn’t match the injury. If someone hurt their wrist, a bill for knee surgery is a red flag.
Upcoding	Billing for a more complex/expensive version of a visit than what actually happened — like billing for a 45-minute specialist consultation when it was really a 10-minute checkup.

How it works, step by step

- A pile of realistic-but-fake bills gets created.** A program generates thousands of synthetic workers’ comp bills. Most are “clean” (no problems); about 1 in 5 has exactly one of the four problems above deliberately built into it, so we always know the right answer for every bill in this practice set.
- A model learns to spot the four problems.** That practice set is used to train a machine learning model — it looks at thousands of examples of “problem” bills and “clean” bills and learns the patterns that tell them apart.
- New bills get scored.** Once trained, the model can look at a bill it’s never seen and give a probability for each of the four problems: how likely is it that *this* bill has *that* issue?
- If something’s flagged, we can point to the exact reason.** Separately from the ML model, this project also has a straightforward rule-checker (plain code, not AI) that can look at a flagged bill and say precisely which billing code caused the flag. That’s the fact that grounds everything else.
- Optionally, an AI writes a plain-English explanation.** If you ask for it, the system looks up the actual rule text that was broken and asks an AI language model to turn it into a readable explanation specific to that bill — not a generic definition, but “here’s what happened on *this* bill and why it matters.”
- All of it is wrapped in a small web app.** You can type a bill ID into a simple chat-style page and see the result immediately, or hit the same functionality programmatically (as an API) from other software.

Is any of this real?

The bills, patients, providers, and dollar amounts are all invented — none of it corresponds to any real person or real claim. What *is* real is the rulebook: the actual CPT/HCPCS procedure codes, ICD-10-CM diagnosis codes, NCCI “don’t bill these together” pairs, and CMS’s per-day billing limits (MUE tables) are the genuine, publicly published coding rules used in the real world. The fake bills were generated so that they realistically obey (or deliberately break) those real rules — that’s what makes the practice data useful for

training something meant to catch real patterns, without using any real, private data to do it.

The tools used, and why each one

Tool	What it's for	Why this one
Python	The programming language everything is written in.	The standard choice for data science and machine learning — huge ecosystem of libraries for exactly this kind of work.
pandas / NumPy	Organizing and crunching the bill data into a table the model can learn from.	The default, well-tested toolkit for this in Python — no need to reinvent it.
scikit-learn	The framework used to build and evaluate the machine learning model.	Reliable, well-documented, and includes almost every standard ML building block needed here.
XGBoost	The specific machine learning algorithm that makes the actual predictions.	A gradient-boosted tree model — consistently one of the strongest performers on exactly this kind of structured, tabular data (rows and columns, not images or text), and fast to train.
MLflow	Keeps a record of every model that gets trained — its settings, its scores, which version is “the” model.	Without it, you’d be tracking “which model performed best” in your head or a spreadsheet. This makes it reproducible and auditable instead.
A local SQLite database	Stores the reference rule tables (the real coding rules) in a simple, queryable format.	Zero setup, no separate server to run — a single file is enough for reference data this size.
LangChain	The toolkit that wires together “look up the right rule” + “ask an AI to explain it” into one pipeline.	Handles a lot of the repetitive plumbing (talking to the vector database, formatting prompts, calling the AI model) so that logic doesn’t have to be written from scratch.
Chroma (a vector database)	Stores the rulebook’s text in a form that can be searched by <i>meaning</i> , not just exact keywords.	Needed for the two problem types that don’t reduce to a single lookup (causality mismatch, upcoding) — it finds the rule whose <i>meaning</i> best matches the situation, which a plain keyword search can’t reliably do.
FastEmbed	Turns rule text into the numeric form (embeddings) the vector database searches over.	It runs entirely on your own machine for free — no per-use cost, unlike a hosted embedding API — which matters because indexing happens often during development.
Azure OpenAI	The hosted AI model that actually writes the plain-English explanation.	This is the one step in the whole project that genuinely benefits from a large, strong general-purpose language model rather than something that can run locally — turning a dry rule citation into a natural explanation is a language task, not a lookup task.
FastAPI	The web framework that exposes everything (the model, the explainer) as a proper web service other software can talk to.	Modern, fast, and it auto-generates interactive API documentation for free, which makes the service much easier to test and demo.
Uvicorn	The actual server process that runs the FastAPI app and listens for requests.	The standard, high-performance way to run a FastAPI app.
Plain HTML/CSS/JavaScript	The simple chat-style web page you actually click around in.	The page is simple enough that a full front-end framework (React, etc.) would be overkill — a single self-contained file keeps it easy to understand and deploy.
Docker	Packages the whole system — code, trained model, database, dependencies — into one self-contained unit that runs identically on any computer.	Without it, “install this on a new machine” means manually recreating a specific Python setup and hoping nothing differs. Docker turns that into one command.

Two design decisions worth calling out

Why an AI explanation isn't just generated for free every time. Calling a hosted AI model costs real money per call, even if it's a small amount. So this project deliberately makes that the *only* part of the system with a real cost, and makes it strictly opt-in — nothing calls the paid AI model automatically just because a bill got flagged. Everything else (spotting the issue, pinpointing which billing code caused it) is done for free with local code and a locally-run model.

Why the AI's explanation is "grounded" instead of just asked to explain things on its own. Language models can sound confident while being wrong. Instead of asking the AI "why might this bill be a problem?" and hoping it gets the specifics right, this project first uses plain, deterministic code to find the *exact* rule that was actually broken, and only then asks the AI to explain *that specific, already-confirmed fact* in plain language. If the rule-checker can't confirm a violation the model flagged, the system says so honestly instead of generating an explanation for something that may not have actually happened.

Where to go for more detail

This overview stays intentionally simple. For the technical version of any of this:

- [README.md](#) — full setup and run instructions.
- [docs/DEMO.md](#) — a hands-on walkthrough of the running app.
- [data-generation/DATA_CARD.md](#) — exactly how the synthetic bills are built.
- [models/MODEL_FRAMING.md](#) and [models/EVALUATION.md](#) — the machine learning details and honest results.
- [agent/RAG_AGENT.md](#) — how the AI explanation feature works under the hood.